

1. Opdrachtbeschrijving:

Voor deze bachelorproef wordt een IoT-systeem ontwikkeld. Een ESP32-microcontroller leest de sensordata en verwerkt deze periodiek tot gestructureerde datapakketten. Deze pakketten worden geëncrypteerd en gehasht, wat maximale beveiliging en integriteit van de payload garandeert. In dit systeem wordt een hash van het datapakket gegenereerd en toegevoegd aan het uiteinde van het pakket. Aan de ontvangende zijde wordt een hash berekend op basis van het ontvangen datapakket (zonder de toegevoegde hash). Deze nieuwe hash wordt vervolgens vergeleken met de oorspronkelijke hash om te controleren of de ontvangen data authentiek en onveranderd is. Dit proces fungeert als een vorm van authenticatie, waarbij wordt gegarandeerd dat de originele data correct is ontvangen.

De gegevens worden via Bluetooth Low Energy (BLE) verzonden naar een Android-smartphone. De Android-smartphone fungeert als gateway tussen de ESP32 en de server. De data wordt eerst geëncrypteerd ontvangen door de app en vervolgens doorgestuurd naar een Linux-server. De server decrypteert de gegevens, slaat deze op in een database en visualiseert de data in een dashboard. Om de beveiliging te versterken en te voorkomen dat de Android-app een zwakke schakel wordt, wordt de data in de app niet gedecrypteerd. In plaats daarvan fungeert de app als een proxyserver die de versleutelde data 1-op-1 doorstuurt van de ESP32 naar de server. De Android-app communiceert met de server om gegevens op te vragen en instellingen aan te passen, zonder zelf betrokken te zijn bij het decryptieproces. Pas nadat de server de data heeft gedecrypteerd, wordt de gedecrypteerde data op de Android-app weergegeven. Hierdoor blijft de versleutelde data volledig beschermd tijdens de transmissie en verwerking.

Dit systeem combineert de kracht van hardware-gebaseerde encryptie, een gebruiksvriendelijke Android-app, en de robuustheid van een Linux-serverarchitectuur.

2. Stand van zaken:

De volgende elementen zijn uitgevoerd:

- 1) Het opstellen van een Gantt-diagram om een duidelijk schema te creëren.
- 2) Onderzoek naar encryptie en hashing, met de selectie van de mbedtls-bibliotheek voor hardware-gebaseerde encryptie op de ESP32.
- 3) Programmeren van de ESP32 met de Arduino IDE, inclusief het uitlezen van sensordata, het encrypteren met AES-128-CBC, hashen met SHA-256, en het verzenden via BLE.
- 4) Ontwikkeling van de Android-app in Android Studio, met implementatie van de GATT-structuur voor communicatie met de ESP32.
- 5) Voorbereiding van placeholders voor de serverimplementatie en datavisualisatie als onderdeel van de projectstructuur.

3. Verdere planning:

Het verzamelen van aanvullende bibliografie is essentieel om bestaande methodologieën en standaarden op het gebied van IoT-beveiliging, BLE-communicatie en serverarchitecturen grondig te bestuderen, zodat de implementatie kan worden geoptimaliseerd. De sensoren moeten gekalibreerd worden om nauwkeurige data te garanderen.

De Android-app vereist aanvullende functionaliteiten en verder debuggen. De integratie met de serverimplementatie is cruciaal omdat de Android-app afhankelijk is van de server voor zowel het ophalen van gedecrypteerde gegevens als het aanpassen van instellingen, zonder zelf decryptie uit te voeren. Databeveiliging en gegevensoverdracht moeten verder worden uitgewerkt. Daarnaast is het optimaliseren en valideren van de payloadlengte cruciaal voor een robuust systeem.

Verder moet de dashboardvisualisatie worden opgezet en moet de end-to-end dataverwerking worden opgezet. Tot slot moet het eindverslag worden geschreven om het volledige proces en de resultaten van het project te documenteren.

Gantt-diagram: [planning bachelorproef manuel mohammed - IoT sensor rapportering via BLE en Android - Google Sheets](#)

Github-repository: <https://github.com/Texas433/Bachelorproef>